

# Hoja de ejercicios - Nebula project

## Construir Nebula con JS

### Objetivo general

- Identificar la sintaxis básica de JS

### Contexto

### Bloque 1. Preparar los datos de la web

#### Ejercicio 1. Crear los datos generales del proyecto

**Objetivo:** Organizar la información necesaria para construir la web utilizando los tipos de datos y las estructuras que resulten más adecuadas en cada caso.

**Tiempo estimado:** 15 minutos.

**Enunciado:**

Vas a preparar en main.js toda la información necesaria para construir la web de Estudio Nébula.

En este ejercicio no debes mostrar todavía nada en el DOM.

Tu tarea consiste en guardar correctamente los datos que se indican a continuación para poder reutilizarlos más adelante al generar la web de forma dinámica. Debes decidir en cada caso qué tipo de dato y qué estructura son los más adecuados según la información que se quiere representar.

#### 1.1. Datos generales del estudio

Guarda la siguiente información:

- Nombre del estudio: Estudio Nébula

#### 1.2. Navegación principal

Guarda la información necesaria para construir los enlaces del menú que se insertarán dentro de #menu-nav:

- Inicio → #inicio
- Servicios → #servicios
- Proyectos → #proyectos
- Contacto → #contacto

### 1.3. Contenido principal del hero

Guarda la información necesaria para construir dentro de #hero-texto:

- texto principal: Transformamos ideas digitales
- palabra destacada: digitales
- descripción: Diseñamos y desarrollamos soluciones web a medida que impulsan tu negocio hacia el futuro con tecnología de vanguardia.
- botón principal: Pedir presupuesto → #contacto
- botón secundario: Ver trabajos → #proyectos

### 1.4. Caja lateral del hero

Guarda la información necesaria para construir dentro de #hero-caja:

1. icono: ⚡
2. título: Entrega en 7 días
3. subtítulo: Pack básico + revisión

### 1.5. Servicios

Guarda la información necesaria para representar estos servicios en #lista-servicios:

- Diseño Web — icono ■ — Interfaces modernas centradas en la experiencia de usuario y la conversión.
- Growth Hacking — icono ✱ — Estrategias de crecimiento acelerado para startups y nuevos productos. — etiqueta NEW
- Desarrollo — icono <> — Código limpio, escalable y optimizado para motores de búsqueda.

### 1.6. Proyectos destacados

Guarda la información necesaria para representar estos proyectos en #lista-proyectos:

- Redesign E-commerce Moda — Implementación completa de una nueva identidad visual para una marca de ropa sostenible. Aumentamos la conversión en un 40% gracias a una experiencia de checkout simplificada y una carga de imágenes optimizada.
- Dashboard Financiero SaaS — Diseño de interfaz para una plataforma de gestión de activos. El reto principal fue simplificar la visualización de grandes volúmenes de datos en tiempo real sin perder rendimiento.

### 1.7. Nota de proyectos

Guarda la información necesaria para construir dentro de #nota-proyectos:

- etiqueta: NOTA (OVERFLOW)
- al menos dos párrafos de texto suficientemente largos para que tenga sentido el overflow definido en el CSS.

## 1.8. Recursos

Guarda la información necesaria para representar estos recursos dentro de #lista-recursos:

- Kit de UI Gratuito
- Guía de Tipografía
- Checklist SEO 2025

Cada recurso debe tener también un href.

Puedes usar "#" como destino si no quieres definir una URL concreta.

## 1.9. Horario

Guarda la siguiente información para construir #bloque-horario:

- icono: ☰
- título: Horario
- texto principal: L–V: 9:00–14:00
- texto auxiliar: Hora local (Madrid)

## 1.10. Footer

Guarda la siguiente información:

- copyright: © 2025 Estudio Nébula
- enlaces legales:
  - Aviso legal
  - Privacidad

Cada enlace legal debe tener también un href.

Puedes usar "#" como destino si no quieres definir una URL concreta.

### Requisitos:

- No se debe escribir todavía nada en el DOM.
- Se debe crear la información usando variables, arrays y objetos.
- Se deben usar arrays de objetos cuando haya varios elementos del mismo tipo.
- No se deben modificar los valores proporcionados.
- Los nombres de variables deben ser claros y coherentes con el contenido.

## Bloque 2. Generar el logo y la navegación

### Ejercicio 2. Mostrar el nombre del estudio en el logo

**Objetivo:** Insertar texto dinámico en un elemento ya existente del DOM a partir de la información guardada previamente.

**Tiempo estimado:** 8 minutos.

**Enunciado:**

Utiliza la información del nombre del estudio que has preparado en el ejercicio 1 para mostrar ese texto dentro del enlace con id logo.

**Requisitos:**

- Se debe seleccionar el elemento con id logo.
- Se debe insertar únicamente el nombre del estudio.
- No se debe modificar el atributo href del enlace.
- No se debe crear un elemento nuevo para el logo. El contenedor ya existe en el HTML base.

### Ejercicio 3. Generar el menú de navegación

**Objetivo:** Crear varios elementos del DOM de forma dinámica a partir de la información guardada previamente.

**Tiempo estimado:** 15 minutos.

**Enunciado:**

Utiliza la información de navegación que has preparado en el ejercicio 1 para generar dentro de #menu-nav los elementos necesarios para construir el menú completo.

Por cada opción del menú debes crear:

- un elemento li,
- un enlace a.

**Requisitos:**

- Se deben insertar 4 enlaces:
  - Inicio,
  - Servicios,
  - Proyectos,
  - Contacto.
- Se deben respetar estos href:
  - #inicio,
  - #servicios,
  - #proyectos,
  - #contacto.
- Se debe añadir la clase nav-link a cada enlace.
- El resultado se debe insertar dentro del ul con id menu-nav.
- La información utilizada debe salir de los datos preparados previamente, no escribirse directamente en el DOM.

## Bloque 3. Generar el contenido principal del hero

### Ejercicio 4. Construir el bloque hero-texto

**Objetivo:** Crear una estructura HTML completa a partir de la información guardada previamente para una sección de la web.

**Tiempo estimado:** 18 minutos.

**Enunciado:**

Utiliza los datos del contenido principal del hero que has preparado en el ejercicio 1 para generar dentro de #hero-texto los siguientes elementos:

- un h1,
- un span con clase highlight,
- un párrafo con clase descripcion,
- un contenedor div con clase acciones,
- dos enlaces con clases:
  - btn btn-primario,
  - btn btn-secundario.

**Requisitos:**

- El título debe reproducir esta idea: Transformamos ideas digitales, dejando la palabra digitales dentro del span con clase highlight.
- El primer botón debe tener como texto Pedir presupuesto y enlazar a #contacto.
- El segundo botón debe tener como texto Ver trabajos y enlazar a #proyectos.
- Se deben usar las clases existentes en el CSS: highlight, descripcion, acciones, btn y btn-secundario.
- No se debe escribir el HTML del hero manualmente en el archivo base.
- La información utilizada debe salir de los datos preparados previamente.

## Bloque 4. Generar la caja lateral del hero

### Ejercicio 5. Construir el bloque hero-caja

**Objetivo:** Crear un bloque informativo sencillo en el DOM a partir de la información guardada previamente.

**Tiempo estimado:** 12 minutos.

**Enunciado:**

Utiliza la información de la caja lateral del hero que has preparado en el ejercicio 1 para generar dentro de #hero-caja los siguientes elementos:

- un span con clase hero-icono,
- un párrafo con clase hero-titulo,
- un párrafo con clase hero-subtexto.

**Requisitos:**

- Se debe respetar este contenido:
  - icono ⚡ ,
  - texto Entrega en 7 días,
  - texto Pack básico + revisión.
- Se deben usar las clases hero-icono, hero-titulo y hero-subtexto.
- No se deben crear elementos fuera del contenedor #hero-caja.
- La información utilizada debe salir de los datos preparados previamente.

## Bloque 5. Generar las tarjetas de servicios

### Ejercicio 6. Crear una tarjeta de servicio

**Objetivo:** Construir un componente reutilizable mediante una función.

**Tiempo estimado:** 15 minutos.

**Enunciado:**

Crea una función llamada `crearTarjetaServicio(servicio)` que reciba un objeto servicio y devuelva un elemento article con clase tarjeta.

Dentro del artículo debe incluir:

- un span para el icono,
- un h3 para el nombre del servicio,
- un p para la descripción.

**Requisitos:**

- La función debe devolver el elemento creado.
- No debe insertarlo todavía en el DOM.
- La función debe estar preparada para reutilizarse con distintos servicios.
- La estructura debe ser compatible con las clases del CSS para `.tarjeta`.

### Ejercicio 7. Pintar todos los servicios

**Objetivo:** Recorrer un array de objetos y renderizar varios elementos repetidos.

**Tiempo estimado:** 15 minutos.

**Enunciado:**

Usa la función del ejercicio anterior para generar dentro de `#lista-servicios` las tarjetas de todos los servicios del estudio.

**Requisitos:**

- Se deben insertar estos 3 servicios:
  - Diseño Web,
  - Growth Hacking,
  - Desarrollo.
- Se debe añadir la etiqueta NEW solo al servicio Growth Hacking, usando un span con clase nuevo.

- El contenedor de destino es #lista-servicios.
- Se deben respetar las clases visuales ya existentes para la sección de tarjetas.

## Bloque 6. Generar los proyectos destacados

### Ejercicio 8. Crear un artículo de proyecto

**Objetivo:** Construir un componente de proyecto reutilizable con DOM.

**Tiempo estimado:** 15 minutos.

**Enunciado:**

Crea una función llamada crearProyecto(proyecto) que reciba un objeto proyecto y devuelva un article con clase post.

Dentro del artículo debe incluir:

- un h3,
- un p,
- un enlace con clase post-link.

**Requisitos:**

- La función debe devolver el artículo creado.
- El enlace debe tener como texto Ver caso de estudio →.
- El enlace debe usar el href guardado en los datos del proyecto.
- La estructura debe ser compatible con las clases .post y .post-link del CSS.

### Ejercicio 9. Pintar todos los proyectos

**Objetivo:** Renderizar una lista de proyectos en el DOM.

**Tiempo estimado:** 12 minutos.

**Enunciado:**

Usa la función del ejercicio anterior para insertar dentro de #lista-proyectos todos los proyectos destacados.

**Requisitos:**

- Se deben insertar en este orden:
  - Redesign E-commerce Moda,
  - Dashboard Financiero SaaS.
- El contenedor de destino es #lista-proyectos.
- No se deben escribir los artículos manualmente en el HTML base.

## Bloque 7. Generar la nota de proyectos

### Ejercicio 10. Construir el bloque nota-proyectos

**Objetivo:** Crear un bloque de texto dentro de un contenedor ya existente.

**Tiempo estimado:** 12 minutos.

### Enunciado:

Utiliza los datos preparados en el ejercicio 1 para generar dentro de #nota-proyectos:

- un párrafo con clase nota-etiqueta,
- al menos dos párrafos con contenido de texto.

### Requisitos:

- El primer párrafo debe contener exactamente el texto NOTA (OVERFLOW).
- El contenido debe ser suficientemente largo como para que tenga sentido el comportamiento de overflow definido en el CSS para .nota.
- El bloque debe insertarse dentro del contenedor con id nota-proyectos.
- El texto debe salir de los datos preparados previamente, no escribirse directamente en el DOM.

## Bloque 8. Generar los recursos del aside

### Ejercicio 11. Pintar la lista de recursos

**Objetivo:** Crear una lista de enlaces en un ul ya existente.

**Tiempo estimado:** 12 minutos.

### Enunciado:

A partir del array de recursos, genera dentro de #lista-recursos los elementos li y a necesarios para mostrar todos los recursos del estudio.

### Requisitos:

- Se deben insertar estos 3 recursos:
  - Kit de UI Gratuito,
  - Guía de Tipografía,
  - Checklist SEO 2025.
- Cada recurso debe mostrarse como un enlace dentro de un `li`.
- El contenedor ya existe en el HTML base y tiene id lista-recursos.
- La lista debe seguir siendo compatible con la clase recursos definida en el CSS.

## Bloque 9. Generar el bloque de horario

### Ejercicio 12. Construir el bloque bloque-horario

**Objetivo:** Crear un bloque informativo en el aside a partir de un objeto.

**Tiempo estimado:** 12 minutos.

### Enunciado:

Genera dentro de #bloque-horario los siguientes elementos:

- un span con el icono,
- un h3 con clase aside-titulo,
- un p con clase horario,



- un último p con el texto auxiliar.

#### Requisitos:

- Se debe respetar este contenido:
  - icono ☹,
  - título Horario,
  - texto L–V: 9:00–14:00,
  - texto auxiliar Hora local (Madrid).
- Se deben usar las clases aside-titulo y horario, ya presentes en el CSS.
- El contenido se debe insertar dentro del contenedor con id bloque-horario.

## Bloque 10. Generar el footer

### Ejercicio 13. Insertar el copyright

**Objetivo:** Mostrar texto dinámico en un contenedor ya existente.

**Tiempo estimado:** 6 minutos.

#### Enunciado:

Usa los datos del estudio para mostrar dentro de #copyright el texto del copyright.

#### Requisitos:

- El texto final debe ser © 2025 Estudio Nébula.
- Se debe insertar en el elemento con id copyright.
- No se debe crear un contenedor nuevo para este contenido.

### Ejercicio 14. Generar los enlaces legales

**Objetivo:** Crear enlaces dinámicos dentro del footer.

**Tiempo estimado:** 10 minutos.

#### Enunciado:

A partir del array de enlaces legales, genera dentro de #footer-links los enlaces necesarios del footer.

#### Requisitos:

- Se deben insertar exactamente 2 enlaces:
  - Aviso legal,
  - Privacidad.
- Cada enlace debe usar el href guardado en los datos.
- El contenedor ya existe y tiene id footer-links.
- No se deben crear elementos li, porque #footer-links es un contenedor nav, no una lista.
- Los enlaces deben mantenerse dentro del footer y aprovechar el estilo ya definido para .footer a.

## Bloque 11. Ensamblar la página completa

### Ejercicio 15. Crear funciones de renderizado

**Objetivo:** Organizar el código en funciones reutilizables.

**Tiempo estimado:** 12 minutos.

**Enunciado:**

Agrupar el código de los ejercicios anteriores en funciones independientes, una para cada parte de la web.

Debes crear como mínimo estas funciones:

- renderLogo()
- renderNav()
- renderHeroTexto()
- renderHeroCaja()
- renderServicios()
- renderProyectos()
- renderNota()
- renderRecursos()
- renderHorario()
- renderFooter()

**Requisitos:**

- Cada función debe encargarse de una única parte de la web.
- No debe haber código duplicado innecesario.
- Las funciones deben trabajar sobre los contenedores ya existentes en el HTML base.

### Ejercicio 16. Renderizar la web completa

**Objetivo:** Construir la página completa a partir de datos y funciones.

**Tiempo estimado:** 10 minutos.

**Enunciado:**

Crear una función renderPagina() que llame, en el orden correcto, a todas las funciones de renderizado anteriores.

Después, ejecutar renderPagina() para que la página se construya completamente al cargar el script.

**Requisitos:**

- La web final debe mostrar:
  - logo,
  - navegación,
  - hero,
  - servicios,
  - proyectos,

- nota,
  - recursos,
  - horario,
  - footer.
- El resultado debe respetar la estructura del HTML base y reutilizar las clases del CSS ya definidas.

## Bloque 12. Ampliación opcional

### Ejercicio 17. Volver a renderizar una sección

**Objetivo:** Actualizar una parte del DOM a partir de nuevos datos.

**Tiempo estimado:** 12 minutos.

**Enunciado:**

Añade un cuarto servicio al array de servicios y vuelve a renderizar solo la sección de servicios.

**Requisitos:**

- Antes de volver a pintar la sección, se debe vaciar el contenido de #lista-servicios.
- Después se deben insertar de nuevo todas las tarjetas, incluida la nueva.
- No se debe recargar la página manualmente.
- La actualización se debe hacer reutilizando la función de renderizado de servicios.